MERU UNIVERSITY OF SCIENCE AND TECHNOLOGY

SCHOOL OF ENGINEERING AND ARCHITECTURE

DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING

BACHELOR OF TECHNOLOGY IN ELECTRICTRAL AND ELECTRONIS ENGINEERING

TITLE: LAB REPORT: BINARY MODULATION AND DEMODULATION (PYTHON IMPLEMENTATION)

UNIT CODE: 3472

Description: Digital Communications

https://github.com/joshuawambua/digitalcommunications.git

Group members

1. *Joshua Wambua EG209/109705/22*
2. *Nason Alfred EG209/111030/22*
3. *Wilkister Kwamboka -EG209/109739/22*

INSTRUCTOR: DR. JOSEPH MUGURO

DATE:20/04/2026

Table of Contents

1. Introduction 1
1.1 Objectives 1
2. Tools Used 1
3. Results 5
3.1 Message Signal 5
3.2 BPSK Modulated Signal 6
3.3 BPSK Signal with Noise 6
3.4 Power Spectral Density (PSD) 7
3.5 Constellation Diagram 7
3.6 M-FSK Signal 8
3.7 M-FSK Signal with Noise 8
4. Discussion 9
5. Conclusion 9
6. References 10

1. Introduction

Digital communication systems transmit information using discrete signals such as binary data (0s and 1s). To send this data over a channel, modulation techniques are used to convert the digital signal into a form suitable for transmission. In this experiment, two modulation schemes were studied:

a. Phase Shift Keying (PSK)
b. M-ary Frequency Shift Keying (M-FSK)

The objective was to generate a random binary signal, modulate it using PSK and M-FSK, add noise to simulate a real communication channel, and then analyze the signals using plots such as waveform graphs, power spectral density, and constellation diagrams.

1.1 Objectives

i. To generate a random binary message signal and select appropriate system parameters such as bit rate and carrier frequency
ii. To implement and analyze Binary Phase Shift Keying (BPSK) and M-ary Frequency Shift Keying (M-FSK) modulation techniques
iii. To simulate the effect of noise on transmitted signals and perform demodulation for signal recovery
iv. To evaluate system performance using waveform plots, Power Spectral Density (PSD), constellation diagrams, and comparison of BPSK and M-FSK under noisy conditions

2. Tools Used

The following tools and libraries were used:

i. **Python** – main programming language used for simulation
ii. **NumPy** – for numerical computations and signal generation
iii. **Matplotlib** – for plotting graphs and visualizing signals
iv. **SciPy (signal module)** – for computing Power Spectral Density (PSD)

The simulation was executed in VS Code.

The python code below was executed in VS code using respective python libraries to obtain the results which are shown afterwards. An equivalent of a the MatLab script can be accessed from the GitHub repository provided.

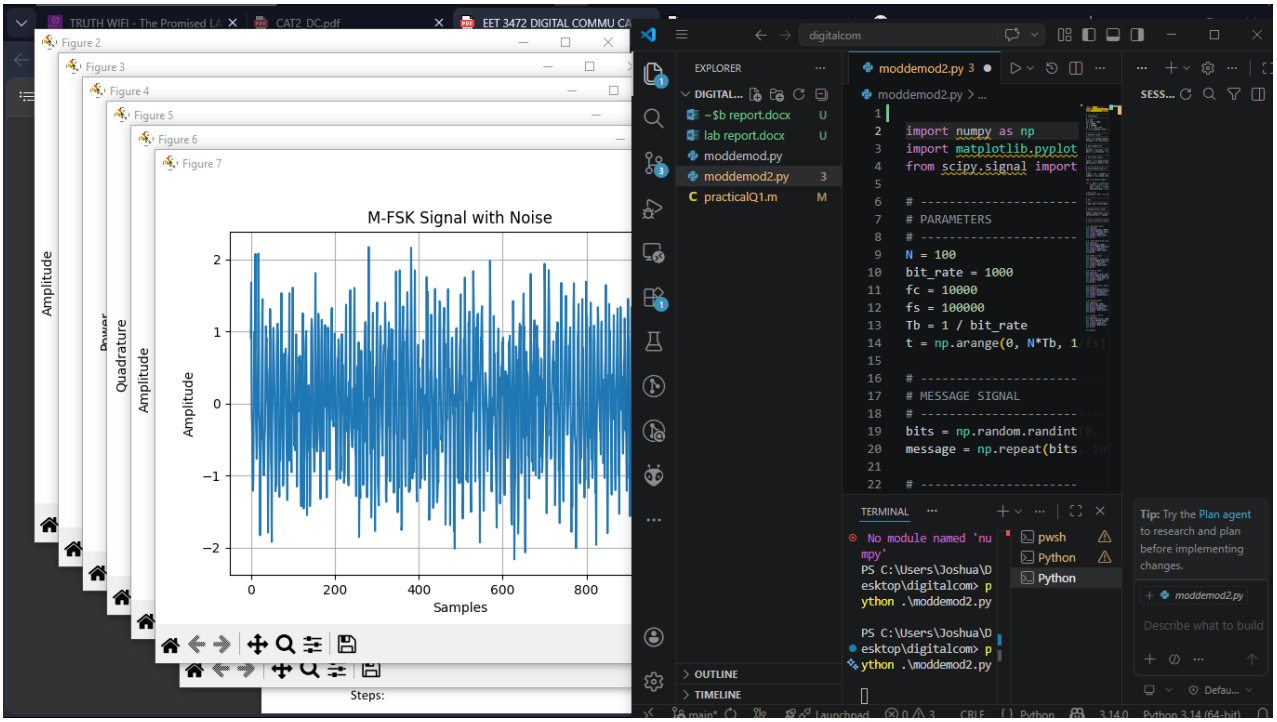
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import welch

# ----------------------------
# PARAMETERS
# ----------------------------
N = 100                  # number of bits
bit_rate = 1000          # 1 kbps
fc = 10000               # carrier frequency (10 kHz)
fs = 100000              # sampling frequency (100 kHz)
Tb = 1 / bit_rate
t = np.arange(0, N*Tb, 1/fs)

# ----------------------------
# MESSAGE SIGNAL
# ----------------------------
bits = np.random.randint(0, 2, N)
message = np.repeat(bits, int(fs*Tb))

# ----------------------------
# BPSK MODULATION
# ----------------------------
carrier = np.cos(2 * np.pi * fc * t)
bpsk = (2 * message - 1) * carrier

# ----------------------------
# ADD NOISE (AWGN)
# ----------------------------
noise = np.random.normal(0, 0.5, len(bpsk))
bpsk_noisy = bpsk + noise

# ----------------------------
# M-FSK MODULATION (M = 4)
# ----------------------------
M = 4
freqs = [fc + i*1000 for i in range(M)]
symbols = np.random.randint(0, M, N)

fsk = np.zeros_like(t)
```

```python
for i, sym in enumerate(symbols):
    start = int(i * Tb * fs)
    end = start + int(Tb * fs)
    fsk[start:end] = np.cos(2 * np.pi * freqs[sym] * t[start:end])

# Add noise
fsk_noisy = fsk + np.random.normal(0, 0.5, len(fsk))

# ----------------------------
# PSD
# ----------------------------
f_psd, psd = welch(bpsk, fs)

# ----------------------------
# DEMODULATION (BPSK)
# ----------------------------
demod = bpsk_noisy * carrier
recovered_bits = (demod > 0).astype(int)

# ----------------------------
# PLOTS (SEPARATE FIGURES)
# ----------------------------

# 1. Message Signal
plt.figure()
plt.plot(message[:1000])
plt.title("Message Signal")
plt.xlabel("Samples")
plt.ylabel("Amplitude")
plt.grid()

# 2. BPSK Modulated Signal
plt.figure()
plt.plot(bpsk[:1000])
plt.title("BPSK Modulated Signal")
plt.xlabel("Samples")
plt.ylabel("Amplitude")
plt.grid()

# 3. BPSK + Noise
plt.figure()
plt.plot(bpsk_noisy[:1000])
plt.title("BPSK Signal with Noise")
plt.xlabel("Samples")
plt.ylabel("Amplitude")
```

```
plt.grid()

# 4. PSD of BPSK
plt.figure()
plt.semilogy(f_psd, psd)
plt.title("Power Spectral Density (BPSK)")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Power")
plt.grid()

# 5. Constellation Diagram (BPSK)
plt.figure()
plt.scatter(bpsk_noisy[:2000], np.zeros(2000))
plt.title("Constellation Diagram (BPSK)")
plt.xlabel("In-phase")
plt.ylabel("Quadrature")
plt.grid()

# 6. M-FSK Signal
plt.figure()
plt.plot(fsk[:1000])
plt.title("M-FSK Signal (M=4)")
plt.xlabel("Samples")
plt.ylabel("Amplitude")
plt.grid()

# 7. M-FSK + Noise
plt.figure()
plt.plot(fsk_noisy[:1000])
plt.title("M-FSK Signal with Noise")
plt.xlabel("Samples")
plt.ylabel("Amplitude")
plt.grid()

plt.show()
```

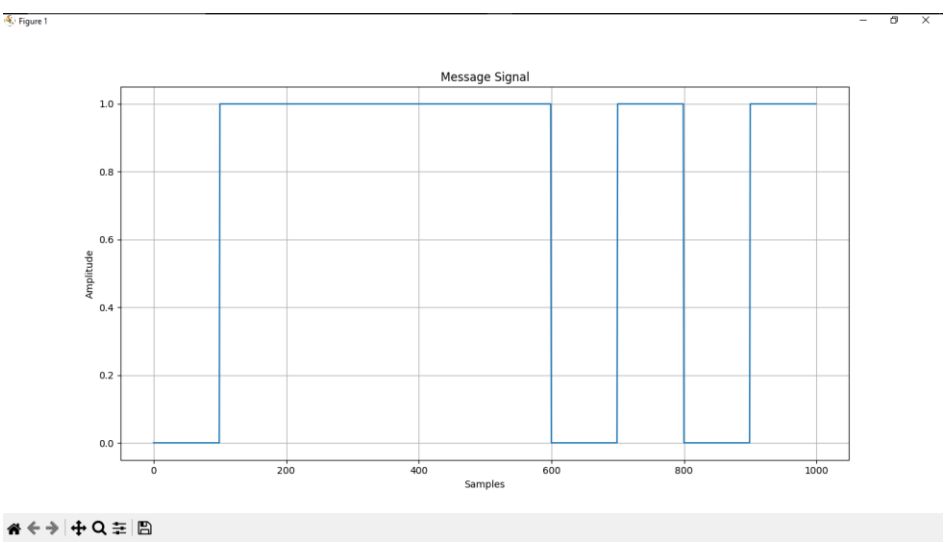
3. Results

The screenshot below shows the code in VS code and the produced figures.



3.1 Message Signal

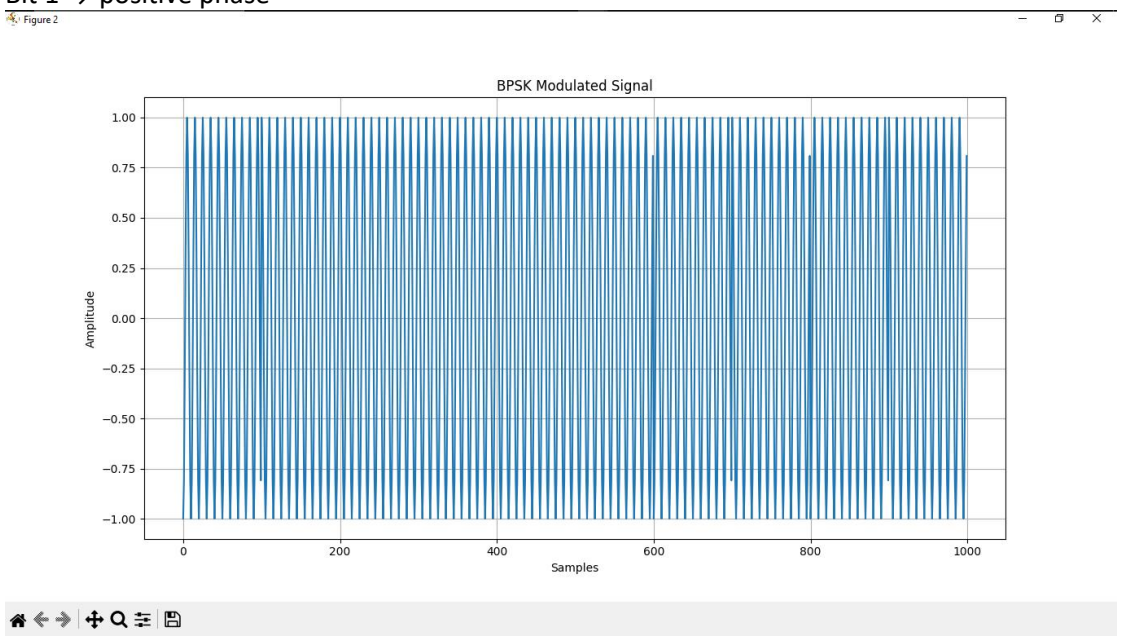
A random binary signal was generated and represented as a digital waveform.



3.2 BPSK Modulated Signal

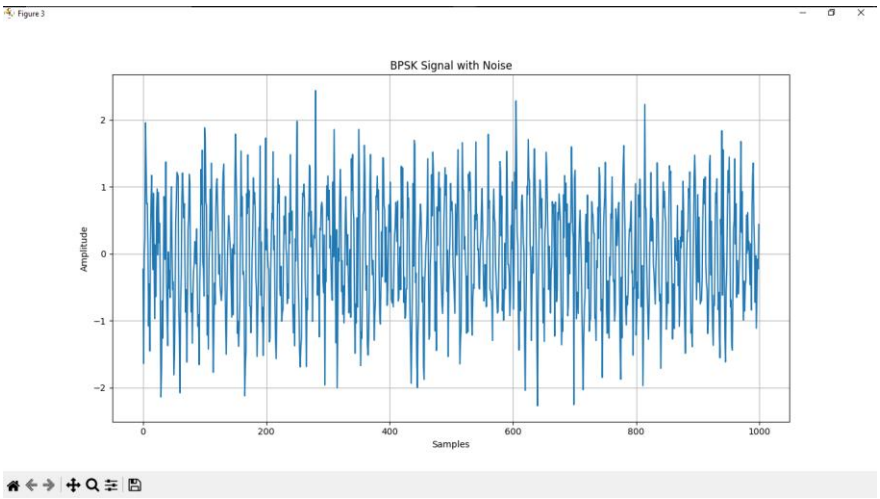
The binary signal was modulated using BPSK, where:

- Bit 0 → negative phase
- Bit 1 → positive phase



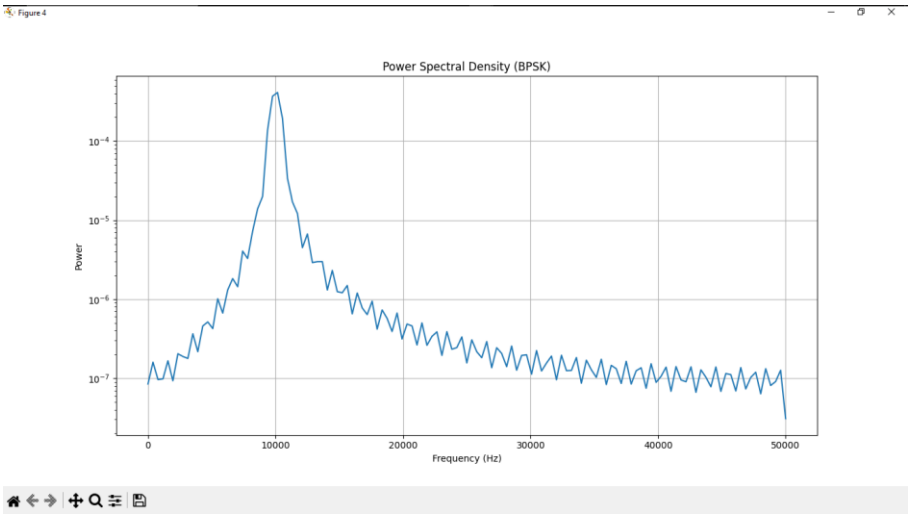
3.3 BPSK Signal with Noise

Noise was added to simulate real-world transmission conditions. The signal becomes distorted but still recognizable.



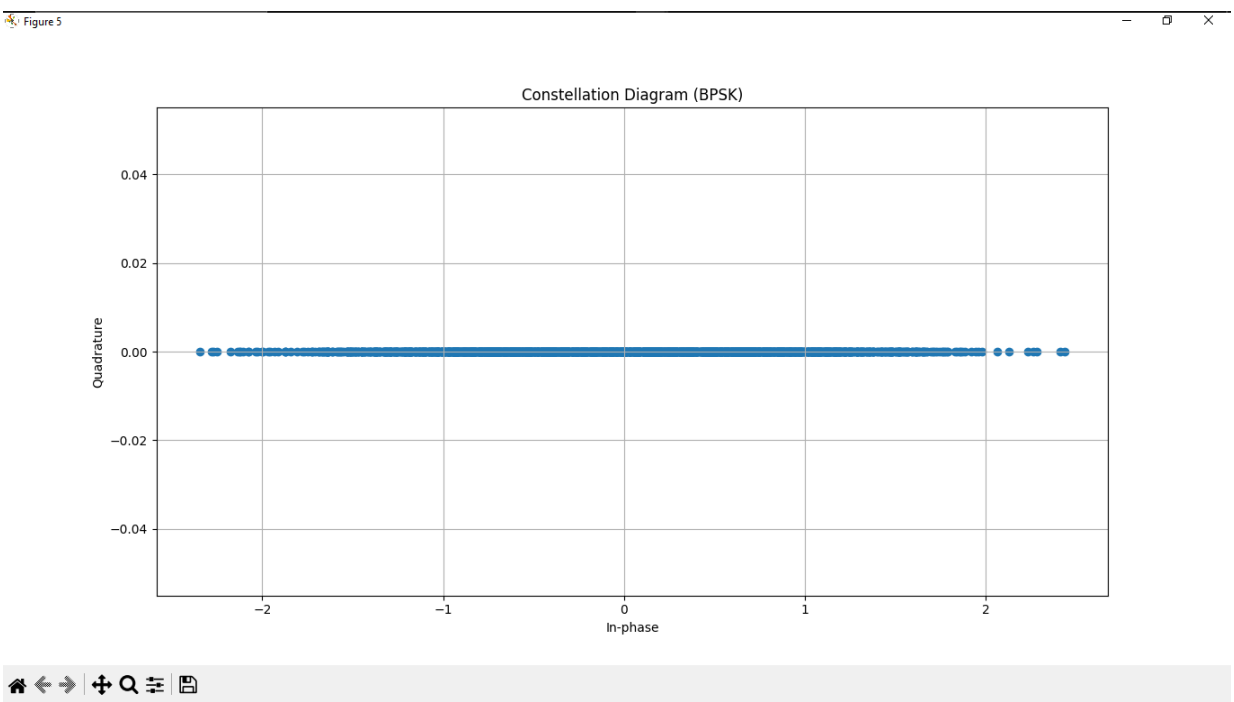
3.4 Power Spectral Density (PSD)

The PSD shows how the signal power is distributed over frequency. The BPSK signal is centered around the carrier frequency.



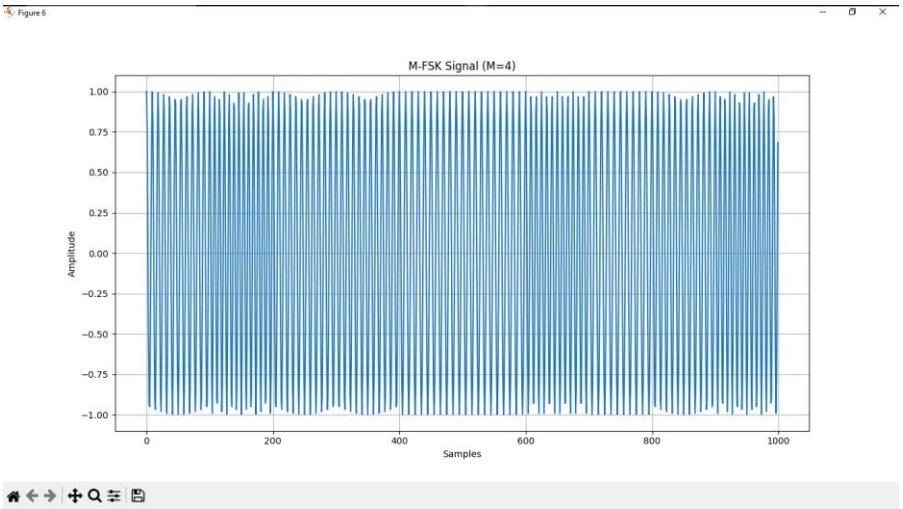
3.5 Constellation Diagram

The constellation diagram shows two distinct points representing binary symbols. Noise causes spreading around these points.



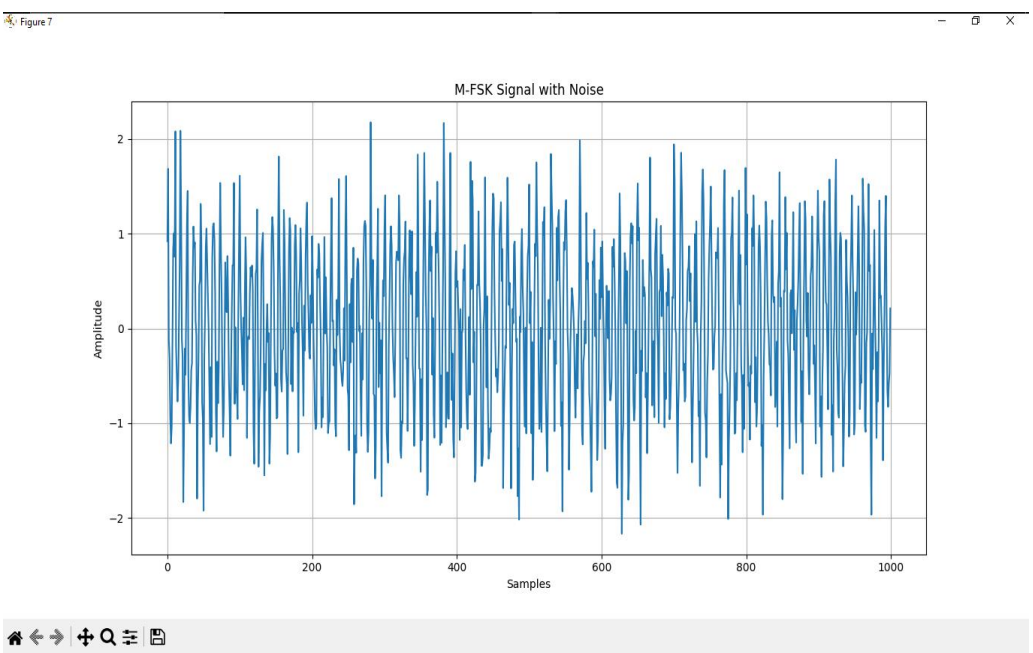
3.6 M-FSK Signal

The signal was modulated using M-FSK (M = 4), where each symbol corresponds to a different frequency.



3.7 M-FSK Signal with Noise

Noise affects the frequency signal, but the different frequency levels can still be identified.



4. Discussion

The simulation results demonstrate how digital modulation schemes behave under both ideal and noisy conditions. In BPSK, the modulation process was observed as a clear phase reversal of the carrier signal corresponding to binary inputs. When noise was introduced, the signal became distorted in amplitude and phase, but the original pattern remained recognizable. The Power Spectral Density showed that most of the signal energy is concentrated around the carrier frequency, confirming that BPSK is bandwidth-efficient. The constellation diagram further illustrated the impact of noise, where the ideal two points spread into clusters, increasing the likelihood of symbol detection errors.

The M-FSK results showed that each symbol is represented by a distinct frequency, making the signal easier to distinguish even in the presence of noise. Unlike BPSK, where phase is critical, M-FSK relies on frequency differences, which are less affected by amplitude variations. This makes M-FSK more robust in noisy environments. However, this improved noise immunity comes at the cost of higher bandwidth usage, since multiple frequencies are required. Overall, the results highlight the trade-off between bandwidth efficiency and noise resistance in digital communication systems.

5. Conclusion

The experiment successfully demonstrated the implementation and analysis of BPSK and M-FSK modulation techniques using Python. It was observed that BPSK is bandwidth-efficient but more sensitive to noise, while M-FSK offers better noise immunity at the expense of increased bandwidth. The use of waveform plots, PSD, and constellation diagrams provided clear insight into signal behavior and the effects of noise. These findings emphasize the importance of selecting an appropriate modulation scheme based on system requirements such as bandwidth and reliability.

6. References

[1] S. Haykin, *Communication Systems*, 5th ed. New York, USA: Wiley, 2009.

[2] B. P. Lathi and Z. Ding, *Modern Digital and Analog Communication Systems*, 4th ed. Oxford, UK: Oxford University Press, 2009.

[3] A. V. Oppenheim and R. W. Schafer, *Signals and Systems*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 1996.

[4] NumPy Documentation. [Online]. Available: https://numpy.org

[5] SciPy Signal Processing Documentation. [Online]. Available: https://scipy.org